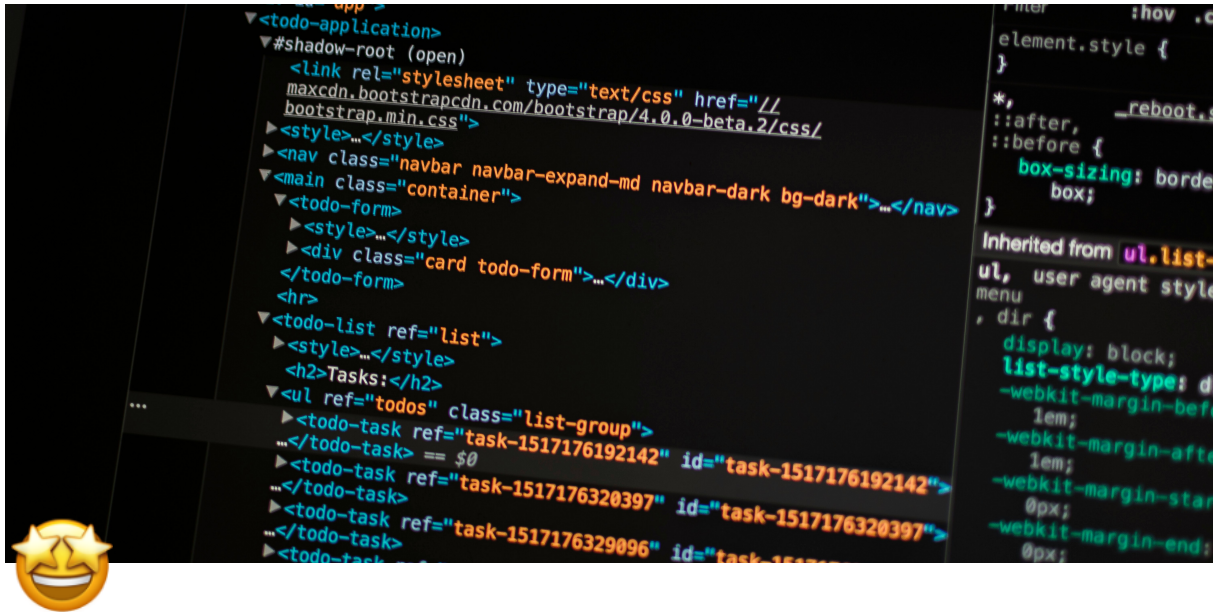




CSS 보충자료 Full Course(센텀직업전문학교 2024)

▼ 11. 웹 애니메이션을 정리해 봅시다 – Transition부터 Web Animation API까지

글의 순서는 아래와 같습니다.

1. CSS Transition
2. CSS Animation
3. Web Animation API

이미 CSS Transition과 CSS Animation에 대해서는 잘 알고 계시다면, Web Animation API를 클릭해서 바로 넘어가시면 됩니다.

Transition

CSS 속성들 중 주로 숫자로 값이 표현되는 속성들(색상 포함)의 값이 변할 때, 중간 과정을 애니메이션으로 만들어 주는 기능. Transition으로 애니메이션이 가능한 속성은 [이 링크](#)를 참고하세요.

Transition을 간단히 사용해 보겠습니다.

영상을 보고 따라해보실 분들은, 아래 HTML 코드를 사용하세요.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>CSS Transition</title>
  <style>
    .box {
      width: 100px;
      height: 100px;
      background: coral;
    }
    .box:hover {

    }
  </style>
</head>
<body>
  <div class="box"></div>
</body>
</html>
```

위 코드에 Transition을 적용해 볼게요. 가장 기본적인 Transition의 사용법입니다.

Transition에서 사용할 수 있는 속성들을 잠깐 확인해 볼게요.

transition-property

Transition을 적용할 CSS 속성 이름을 적어줍니다.

기본값은 all로, 가능한 모든 속성에 Transition을 적용해요.

transition-duration

Transition 효과를 지속(재생)하는 시간.

초(s) 또는 밀리초(ms) 단위를 사용해요.

transition-delay

Transition이 시작되기 전 대기 시간.

초(s) 또는 밀리초(ms) 단위를 사용해요.

transition-timing-function

가속이나 감속을 적용해서, 애니메이션의 느낌, 텐션을 조정해요.

점점 빠르게, 점점 느리게, 점점 빨라지다가 점점 느려지게, 일정한 속도로... 등등, 그런 것들 말이에요.

Transition을 속성마다 따로 적용하고 싶다면, 다른 CSS 속성들과 마찬가지로 쉼표로 구분해서 적으면 됩니다.

속성들을 기억하는 게 어렵다면, 아래의 그림을 머리에 '얼핏' 넣어 두세요. 정확하지는 않아도 '어떻게 가능했었는지' 정도만 기억해두면 사용법은 필요할 때 찾아보면 되니까요. 텍스트 정보 보다 그림으로 기억해 두는게 머리에 더 오래 남을 거예요.

알약 모양에 써 있는 단어 앞에 transition-을 붙여주면, Transition에서 사용 가능한 속성 이름입니다.

자동으로 적절한 값을 적용시키는 auto는 많은 속성들의 기본값인데요, 이 auto는 숫자값이 아니기 때문에 transition이 올바르게 동작하지 않습니다. transition으로 애니메이션 효과를 줄 때는 auto값의 사용은 피해주세요.

Transition은 자동으로 재생되지 않아요. 마우스를 올리든 특정 클래스가 추가되든, 어떤 액션이 있어야 실행됩니다. 페이지가 로드되고 자동으로 재생되는 걸 원한다면, 아래에 있는 CSS Animation을 사용하세요.

Animation

Animation 역시 Transition과 마찬가지로 CSS 값의 변화에 따라 애니메이션 효과를 만들어 주는데요, Transition보다 기능이 더 많습니다. 키프레임 개념이 있어서 애니메이션 중간에 원하는 지점에서 CSS에 변화를 줄 수 있고, 일시정지나 재생 등의 컨트롤도 가능해요.

일단 Animation을 간단히 사용해 볼게요.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>CSS Animation</title>
  <style>
    @keyframes box-ani {
      0% {
        transform: translateX(0);
      }
      35% {
        transform: translateX(400px);
      }
      100% {
        transform: translateX(300px);
      }
    }
  </style>
</head>
</html>
```

```

    }
    .box {
        width: 100px;
        height: 100px;
        background: forestgreen;
        animation: box-ani 2s;
    }
</style>
</head>
<body>
    <div class="box"></div>
</body>
</html>

```

Animation에 있는 속성은 아래와 같아요. 몇몇 속성들은 Transition과 흡사합니다.

animation-name

우리가 직접 지어주는 애니메이션의 이름. 원하는 곳에서 애니메이션을 사용할 때, 이 이름을 사용해요.

@keyframes 원하는이름

animation-duration

애니메이션을 지속(재생)하는 시간.

초(s) 또는 밀리초(ms) 단위를 사용해요.

animation-delay

애니메이션이 시작되기 전 대기 시간.

초(s) 또는 밀리초(ms) 단위를 사용해요.

animation-timing-function

가속이나 감속을 적용해서, 애니메이션의 느낌, 텐션을 조정해요.

점점 빠르게, 점점 느리게, 점점 빨라지다가 점점 느려지게, 일정한 속도로... 등등, 그런 것들 말이에요.

animation-iteration-count

애니메이션의 반복 횟수를 숫자로 지정.

무한 반복을 원하면 infinite라고 써주면 돼요.

animation-direction

애니메이션의 재생 방향.

- **normal** 정방향 재생. 타임라인 0%(from)에서 100%(to) 방향으로.
- **reverse** 역방향 재생. 타임라인 100%(to)에서 0%(from) 방향으로.
- **alternate** 정방향, 역방향, 정방향, 역방향... 으로 번갈아 가면서 재생.
- **alternate-reverse** 역방향, 정방향, 역방향, 정방향... 으로 번갈아 가면서 재생.

animation-fill-mode

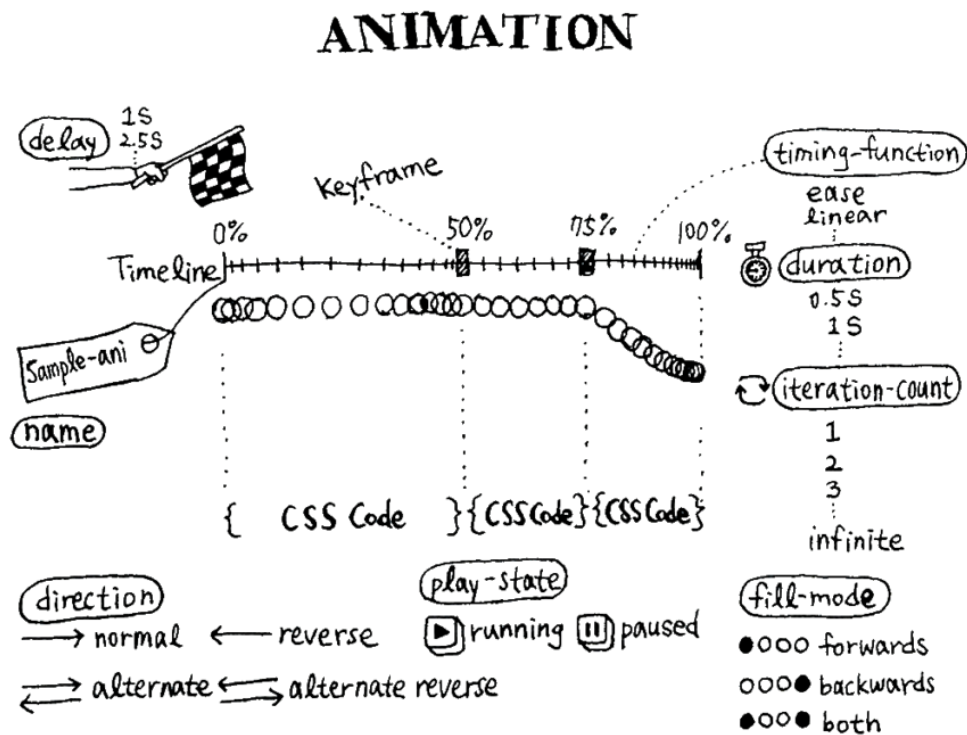
애니메이션이 재생되고 있지 않은 상태(시작 전, 후)에 어떤 스타일을 적용해줄 지 결정하는 속성.

- **none** 재생 전, 후 모두 CSS에 선언된 스타일이 적용.
- **forwards** 재생 전에는 CSS에 선언된 스타일, 재생 후에는 타임라인의 100% 지점 스타일이 적용.
- **backwards** 재생 전에는 타임라인의 0% 지점 스타일, 재생 후에는 CSS에 선언된 스타일이 적용.
- **both** 재생 전에는 타임라인의 0% 지점 스타일, 재생 후에는 타임라인의 100% 지점 스타일이 적용.

animation-play-state

애니메이션의 재생 상태.

running이 재생 중, **paused**는 일시 정지 상태.

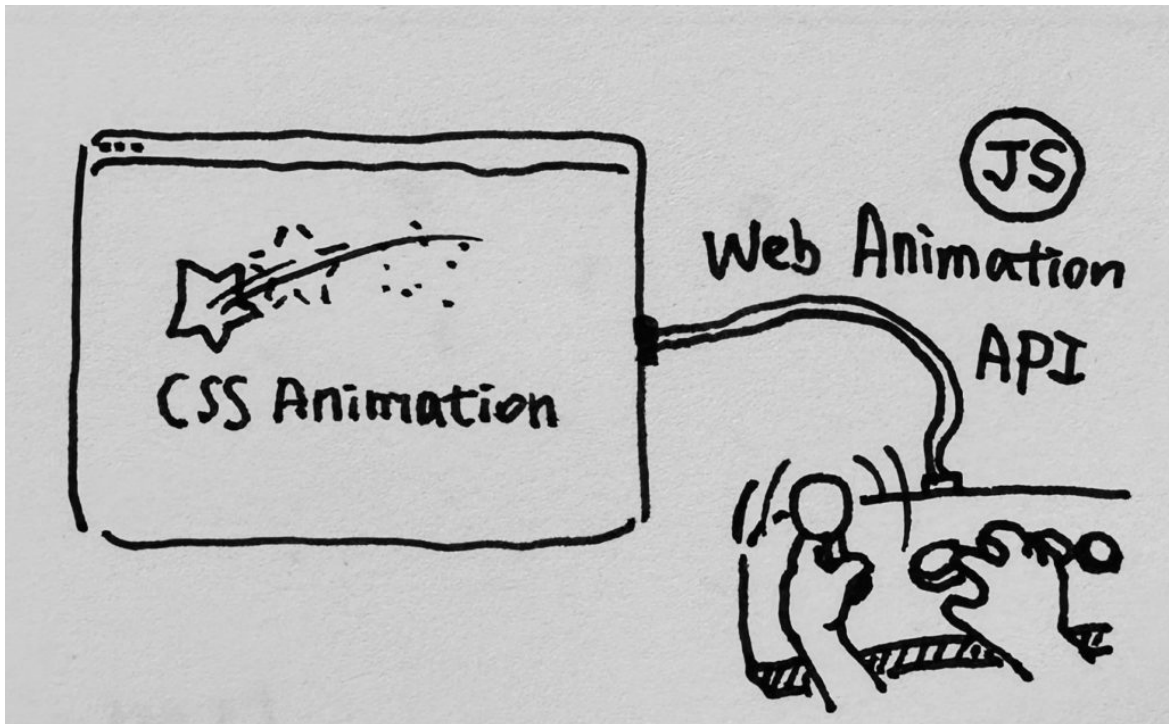


알약 모양에 써 있는 단어 앞에 animation-을 붙여주면, Animation에서 사용 가능한 속성 이름입니다.

Transition과 Animation을 사용할 때는 Transform과 Opacity 두 속성에만 적용하는 것이 성능상 바람직합니다. 특히 이동은 left, top 등의 속성 대신, transform의 translate, translateX, translateY, translate3d를 사용하세요.

Web Animation API

CSS Animation의 구현을 기반으로 설계된 API로, 자바스크립트로 CSS 애니메이션을 제어할 수 있어요.



기존 방식 애니메이션의 단점

CSS를 이용한 애니메이션은 미리 정해진대로만 재생되는 애니메이션을 만드는 데에는 아주 편리하지만, 상황에 대응해서 제어하기가 어려워요. 원하는 시점에 멈춘다든지, 특정한 조건이 만족된 상황에서 재생시킨다든지.. **마음대로 컨트롤 하기가 쉽지 않지요.**

window.requestAnimationFrame을 이용해서 빠른 속도로 반복 실행하는 자바스크립트 애니메이션은 메인 스레드를 점유하기 때문에 성능이 좋기가 어렵고요.

그래서, Web Animation API

Web Animation API는 이 문제들을 해결할 수 있는 **웹 애니메이션 표준 기술(Web Standard)**입니다. 브라우저 자체적으로 최적화를 진행해서 성능이 좋은 CSS 애니메이션의 장점을 살리면서, 자바스크립트로 구현하는 기존 방식의 애니메이션보다 구현도 간편하지요. 사용자의 입력 등에 반응해서 대화형 애니메이션을 구현할 수도 있고, 여러 개의 애니메이션을 동시에 재생하거나 또는 순차적으로 재생할 수도 있어요. 스크롤을 타임라인처럼 이용해서 스크롤 애니메이션을 구현할 수도 있고요.

무엇보다, **외부 라이브러리가 아닌 웹 표준 기술**이기 때문에, 익혀두면 두고두고 사용할 수 있습니다.

2023년 4월 현재, 아직 일부 기능(ScrollTimeline 등)은 구현이 안되어서 폴리필(Poly-fill) 스크립트를 사용해야 합니다. 그치만 그것들도 사용법은 표준 스펙 형태를 따르고 있어요. 그러므로 미래에 표준 스펙에서 구현된 후에도 코딩 자체는 바뀌지 않을 가능성이 크기 때문에, 추후에 폴리필 용으로 사용한 라이브러리를 걷어내기도 수월할 것으로 생각됩니다.

그럼 WAAP을 간단히 사용해 보겠습니다.

영상을 보고 따라해보실 분들은, 아래 HTML 코드를 사용하세요.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Web Animation API</title>
  <style>
    .box {
      width: 100px;
      height: 100px;
      background: dodgerblue;
    }
  </style>
</head>
<body>
  <div class="box"></div>
  <script>
    const box = document.querySelector('.box');

  </script>
</body>
</html>
```

위 코드에 Web Animation API를 사용해 볼게요.

키프레임을 타임라인 특정 지점에 지정할 때는, **offset** 속성을 사용하면 됩니다.

타임라인 0%가 offset 0, 100%가 offset 1입니다. 아래 영상을 봐보세요~

duration(애니메이션 지속 시간) 외에도 CSS Animation에서 사용하던 모든 속성들을 사용할 수 있어요.

아래 표와 영상을 참고하세요.

CSS Animation	Web Animation API
animation-duration	duration

animation-delay	delay
animation-direction	direction
fill-mode	fill
timing-function	easing
iteration-count	iterations

옵션에 사용하는 이름들이 CSS Animation과 좀 다르답니다

사실 여기까지만 한다면, 굳이 WAAPI를 사용할 필요가 없겠죠? CSS로 작성한 애니메이션이랑 효과가 같은데 더 어렵기만 하니까요. WAAPI의 강점인 애니메이션 컨트롤을 해보겠습니다.

이런 식으로 CSS 애니메이션을 제어할 수 있어요. 스크립트를 이용하는 만큼, 각 속성값들을 원하는 값으로 계산해서 적용하는 것도 수월하고요. CSS만 이용해서 작성하는 것과 확실한 차별점이 느껴지죠?

이런 기본 컨트롤을 외에도, 정말 다양한 애니메이션 효과를 만들 수 있습니다.

이렇게 여러개의 <div>를 좋은 성능으로 컨트롤 한다든지..

스크롤 기반의 애니메이션도 처리할 수 있어요.

자세한 스펙은 아래 문서를 참고해 보세요.

https://developer.mozilla.org/en-US/docs/Web/API/Web_Animations_API

웹 애니메이션을 정리해 봅시다 - Transition부터 Web Animation API까지

CSS Transition과 Animation, 그리고 CSS Animation을 기반으로 한 Web Animation API의 기본적인 내용들을 정리해 봅니다. 스크롤 압박이 있지만, 1분 내외의 영상들로 짧막하게 구성해서 보시는 데에 큰 부담은 안가지셔도 될 거예요....

 <https://studiomeal.com/archives/1465>

